알고리즘1 (2026-1)

1. 알고리즘의 설계와 표현

국립금오공과대학교 컴퓨터공학부

컴퓨터공학전공

김 경 수

목차

I. 알고리즘의 정의

II. 알고리즘의 설계와 표현

III. 대표적인 알고리즘 패러다임

학습목표

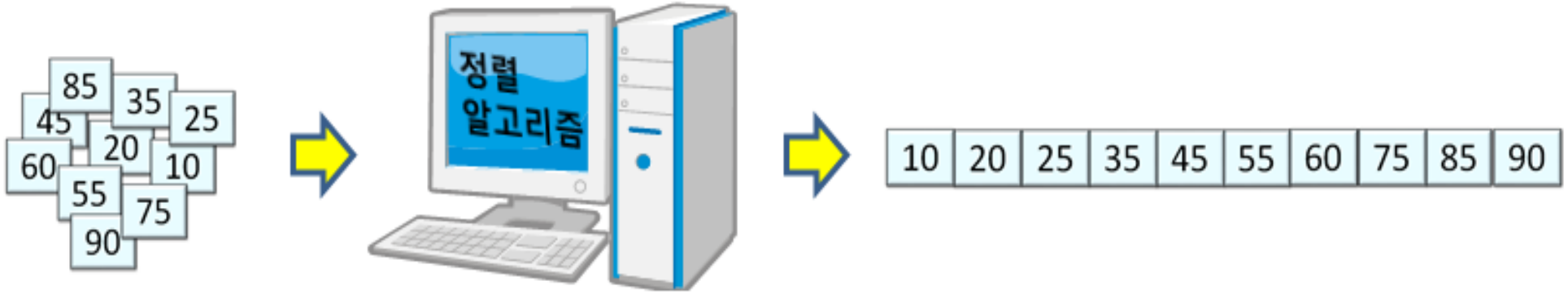
학습목표위주로시험에잘나옴

- 알고리즘의 정의와 일반적 특성, 알고리즘 설계의 개념과 절차에 대하여 이해하고 설명할 수 있다.
- 알고리즘 설계의 원칙을 숙지하고, 실제 알고리즘 설계에 적용할 수 있다.
- 주어진 문제를 해결하기 위한 알고리즘을 자연어와 플로우 차트, 의사 코드 등을 이용하여 작성하고 설계할 수 있다.
- 다양한 알고리즘의 패러다임을 이해하고, 각 패러다임의 대표적인 문제에 대하여 설명할 수 있다.

Ⅰ. 알고리즘의 정의

알고리즘이란 무엇인가?

- 알고리즘이란 **문제를 해결하는 단계적 절차 또는 방법**을 의미한다.
- 여기서 주어지는 문제는 컴퓨터를 이용하여 해결할 수 있어야 한다.
- 알고리즘에는 입력이 주어지고, 알고리즘을 수행한 결과인 해(또는 답)를 출력한다.



알고리즘의 일반적 특성

무조건시험에나옴

- **정확성(Correctness)**

  예외: 근사 알고리즘 (올바른 해 x)
  일정한 조건 -> 정확성 판단 기준

  - 알고리즘은 주어진 입력에 대해 **올바른 해**를 주어야 한다.

- **유효성(Effectiveness)**
  - 알고리즘의 각 단계는 **실제 사람이 또는 컴퓨터가 수행 가능**해야 한다.

- **유한성(Finiteness)**
  - 알고리즘은 **유한 시간 내에 종료**되어야 한다.

알고리즘의 일반적 특성

- **명확성(Definiteness)**
  - ➢ 알고리즘에 포함된 모든 명령어들과 절차는 **정확하고 명료**해야 한다.
- **효율성(Efficiency)**
  - ➢ 주어진 문제를 해결하는 다양한 방법이 있을지라도 보다 **적은 계산량**과 **시간**으로 **효율적으로** 문제를 해결할 수 있는 알고리즘을 고안하는 것이 중요하다.

알고리즘의 일반적 특성

특성적용시험예시

- 아래 버블 정렬 알고리즘의 일반적 특성을 분석해보자.

```
Bubble_sort(A)
{
    for i=1 to n
        for j= n to i+1
            if A[j]<A[j-1]
                swap A[j]<->A[j-1]
    return A
}
```

알고리즘 특성
정확성
유효성
유한성
명확성
효율성

상기 다섯 가지 특성을 모두 만족한다.

Ⅱ. 알고리즘의 설계와 표현

알고리즘의 설계

- **알고리즘이란?**
  - 어떤 일을 해결하기 위한 방법이나 문제를 해결하기 위한 절차 등을 단계적으로 나열한 것
- **알고리즘 설계란?**
  - 주어진 문제를 해결하는 방법을 논리적으로 탐구하는 과정
  - 주어진 문제의 특성과 조건, 제한사항 등을 이해하고, 이를 토대로 문제 해결을 위한 다양한 방법과 절차를 논리적으로 도출하며, 실제 그 결과를 확인하는 일련의 전 과정을 포함

알고리즘 설계의 원칙

- 문제의 정의와 특성을 완벽하게 이해한다.
- 해결하려는 문제와 같은(또는 유사한) 유형의 기존 문제들을 해결하는 알고리즘이 존재하는 지 확인하고, 해당 문제에 적용 가능한지 검토한다.
- 주어진 문제를 해결하는 과정이 복잡한 경우, 여러 개의 작은 단위로 나누어 작성하는 방법을 고려해 본다.
- **[참고]** 복잡한 문제를 해결하는 다양한 방법
  - ✓ 분할 정복법 (Divide-and-conquer strategy)
  - ✓ 동적 계획법 (Dynamic programming)
  - ✓ 상태 공간 탐색 (State-space search strategy) … 등

좋은 알고리즘이란?

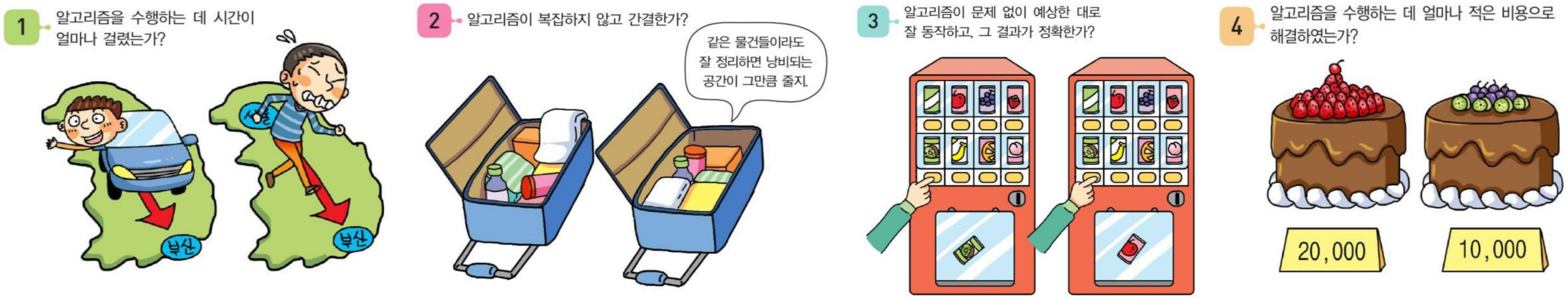
알고있을것

① 알고리즘의 수행 시간이 합리적이다.

② 알고리즘이 복잡하지 않고 간결하다.

③ 알고리즘이 문제없이 예상한대로 잘 동작하고 그 결과가 정확하다.

④ 알고리즘을 수행하는 데 소요된 비용이 적을수록 좋다.



알고리즘의 표현 방법

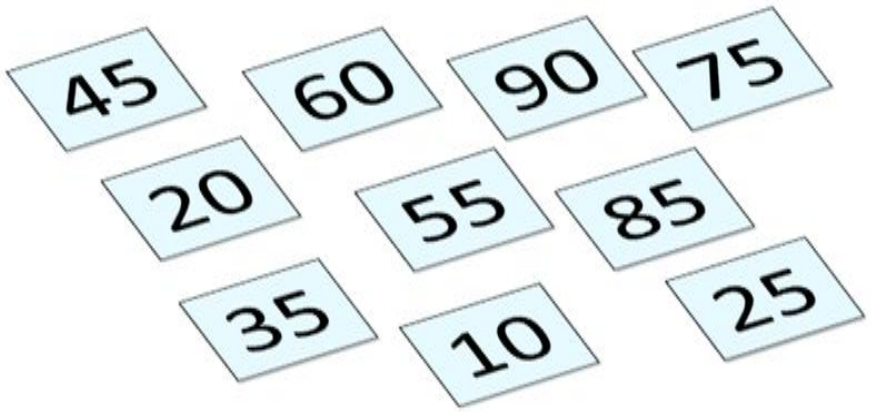
※ 알고리즘은언어와독립적으로설계

- 알고리즘의 형태는 일련의 단계별 절차이므로 보통 말로 서술할 수 있으며, 컴퓨터 프로그래밍 언어로만 표현할 필요는 없음.
- 일반적으로 알고리즘은 프로그래밍 언어와 유사한 **의사 코드(pseudo code)**로 표현함.
  - ➢**이유:** 가독성, 간결성, 정확성
  - ➢의사코드는 프로그래밍 언어 형태와 유사하되, 각 단계의 기능을 모호성 없이 명확하게 표현할 수 있도록 간결해야 함.

문제의 특성 분석
- 제한 사항
- 입 / 출력
- ★주어진 문제가 어떤 알고리즘 패러다임을 쓰는가?

알고리즘의 표현 방법

- (예) N개의 숫자 중에서 최대 숫자를 찾는 방법



< 문제 해결 방법>

① 숫자를 하나씩 비교하면서 지금까지 확인한 숫자 중 가장 큰 숫자를 기억해가며 찾는다.

② 마지막 숫자까지 확인한 후, 현재까지 기억된 가장 큰 숫자가 N개의 숫자 중에서 가장 큰 숫자이다.

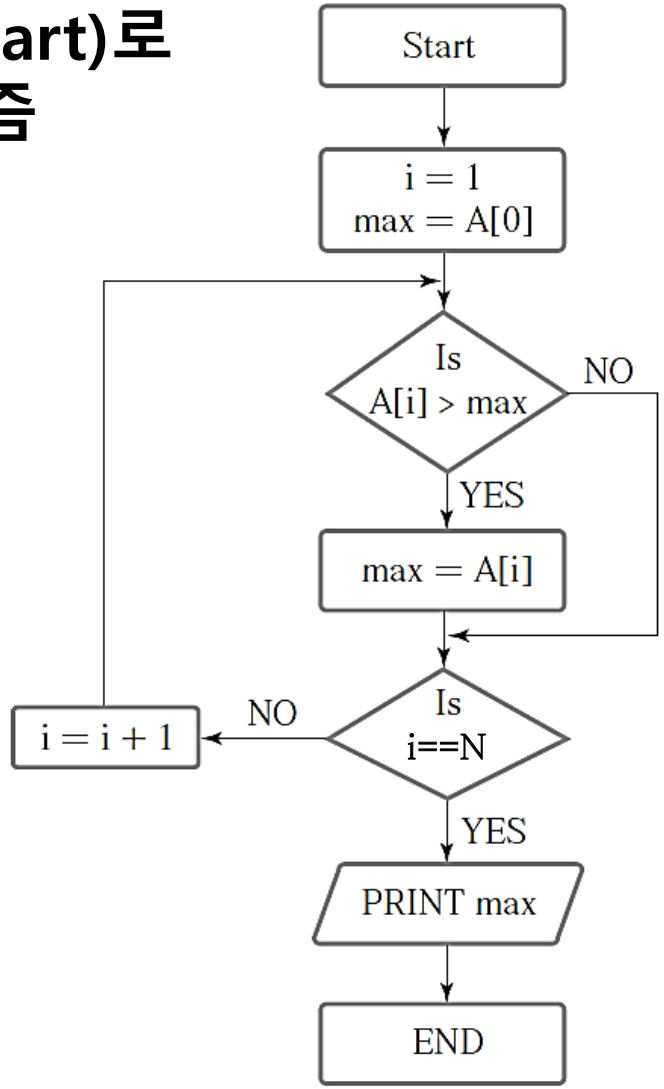
알고리즘의 표현 방법

자연어로 표현한 알고리즘

1. 첫 번째 숫자를 읽고 그 값을 기억해 둔다.
2. 다음 숫자를 읽고, 그 숫자를 기존에 기억해 둔 숫자와 비교한다.
3. 비교 후 큰 숫자를 기억해 둔다.
4. 다음에 비교할 숫자가 남아있으면 "단계 2"로 이동한다.
5. 현재까지 기억되어 있는 숫자가 최대 숫자이다.

알고리즘의 표현 방법

플로우 차트(flow chart)로 표현한 알고리즘



기호	이름	의미
	처리	각종 연산이나 데이터 이동 등의 처리
	연결자	흐름이 다른 곳과 연결되는 입구와 출구
	흐름선	처리의 흐름, 기호를 연결하는 기호
	입 · 출력	데이터의 입력과 출력
	준비	작업의 준비 과정
	터미널	순서도의 시작과 끝을 표시
	정의된 처리	미리 정의된 처리로 실행을 옮길 때 사용
	서류	서류를 매체로 하는 입출력
	수동 입력	콘솔에 의한 입력
	조건	여러 개의 경로 중 하나의 경로 선택

알고리즘의 표현 방법

의사 코드로 표현한 알고리즘

```
Algorithm find_max_value(A[], N):
    if (length(A) < 1) then
        max = A[0]
        for (i = 1 to N)
            if (A[i] > max) max = A[i]
        return max
    else
        error("배열의 길이가 1보다 작음.")
```

알고리즘의 설계와 표현 연습

- **[문제] 1보다 큰 정수 n을 입력받아 n이 소수인지 아닌지 판별하는 프로그램을 작성하시오.**

  ① 상기 문제를 해결하는 방법을 설계하고 자연어로 나타내시오.

  ② 상기 문제를 해결하는 방법을 플로우 차트로 나타내시오.

  ③ 상기 문제를 해결하는 방법을 의사코드로 표현하시오.

[참고]

소수(prime number): 나누어 떨어지는 수가 1과 자기 자신 뿐인 수

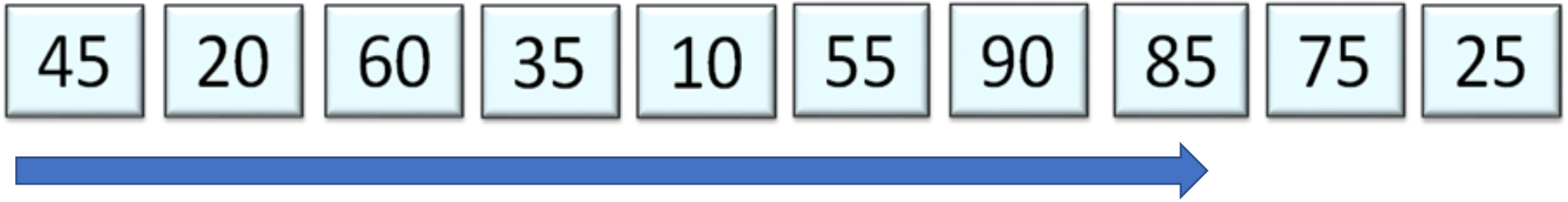
Ⅲ. 대표적인 알고리즘 패러다임

대표적인 알고리즘 패러다임

- **문제의 해결 방식에 따른 알고리즘의 패러다임**
  - 순차 탐색(Sequential Search) 알고리즘
  - 이진 탐색(Binary Search) 알고리즘
  - 그리디(Greedy) 알고리즘
  - 동적 계획(Dynamic Programming) 알고리즘
  - 분할 정복(Divide-and-Conquer) 알고리즘
  - 백트래킹(Backtracking) 및 분기 한정(Branch-and-Bound) 알고리즘
  - 근사(Approximation) 알고리즘 …… 등

임의의 숫자 찾기 문제

- 아래 카드에서 임의의 숫자 “85”를 찾아보자.

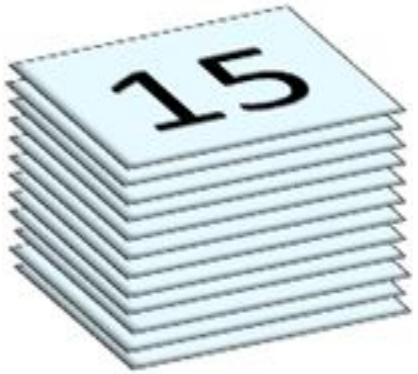


➢ 최대 숫자 찾기처럼 **85**를 기억하고 바닥에 펼쳐진 카드를 차례대로 한 장씩 읽으면서 해당 카드에 적힌 값이 85인지 비교하는 방법으로 “85”를 찾을 수 있다.

- 위와 같은 방법을 **순차 탐색(Sequential Search)**이라 한다.
- 즉, 카드를 한 장씩 차례대로(순서대로) 읽어가면서 탐색하는 방법이다.

임의의 숫자 찾기 문제

- 만약 10장의 카드가 오름차순으로 미리 정렬되어 있다면?
  - (예) 사전, 휴대폰의 전화번호 리스트, 인덱스 등



- 정렬된 카드 뭉치에서 **85** 찾기
  - 15, 20, 25, 35, 45, 55, 60, 75, 85, 90에서 **85**를 **순차 탐색**으로 찾으면

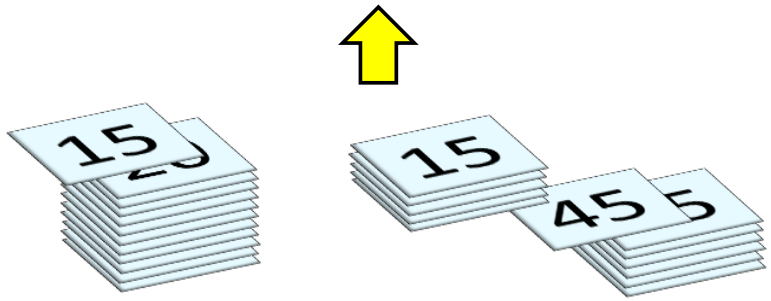
    ➔ 위로부터 8장의 카드를 읽은 후에나 **85**를 찾는다.

임의의 숫자 찾기 문제

➢ 그렇다면, 카드 뭉치가 미리 정렬되어 있는 상황을 어떻게 활용할 수 있을까?

중간에 있는 45 (혹은 55)와 85를 먼저 비교

15, 20, 25, 35, **45**, 55, 60, 75, 85, 90



이진 탐색
(Binary Search)

- 중간 카드 한 장을 읽어 85와 비교하는 방법이 첫 카드부터 읽어 나가는 순차 탐색보다 **훨씬 빠르게** 목표(85)에 도달할 수 있다.

임의의 숫자 찾기 문제

- **이진 탐색(Binary Search)**을 활용하여 숫자 K를 찾는 알고리즘

① 리스트 내 모든 데이터를 오름차순으로 정렬한다.

② 리스트 내의 가운데 위치한 숫자와 K를 비교한다.

- 두 수가 같으면 탐색 성공으로 해당 숫자를 반환하고 알고리즘을 종료한다.
- K가 작으면 가운데 숫자 기준 앞 부분의 리스트에서 단계 ②를 반복한다.
- K가 크면 가운데 숫자 기준 뒷부분의 리스트에서 단계 ②를 반복한다.

③ "탐색 실패"를 반환하고 알고리즘을 종료한다.

동전 거스름돈 문제

- 물건을 사고 거스름돈을 동전으로 받아야 한다면?
  - ➢ 대부분의 경우 **가장 적은 수**의 동전을 받기를 원한다.

- 만약 거스름돈이 730원이라면 받아야 하는 동전의 최소 개수는?
  - ➢ 500원 동전 1개 + 100원 동전 2개 + 10원 동전 3개 = 총 6개

➔ **거스름돈 730원에 대한 최소 동전의 수**

동전 거스름돈 문제

- 어떻게 하면 가장 적은 수의 동전을 찾을까?

→ 가장 큰 액면의 동전부터 차례로

- **(예)** 730원을 거슬러주기 위한 최소 동전의 수는?

① 500원 동전 **1개** 선택

② 남은 230원에 대해 100원 동전 **2개** 선택

③ 남은 30원에 대해 10원 동전 **3개** 선택

④ 총 동전의 수 = 1 + 2 + 3 = **6개** (최소 동전의 수)

동전 거스름돈 문제

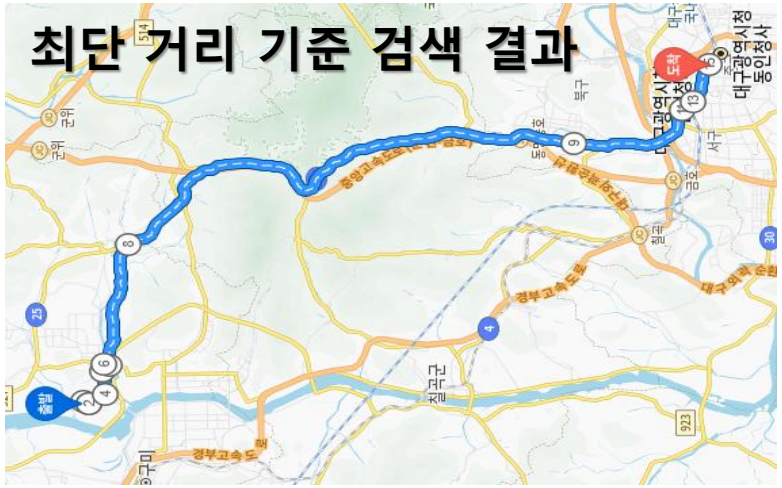
- **어떻게 하면 가장 적은 수의 동전을 찾을까?**
  - ➢ 남은 거스름돈 액수를 넘지 않는 한도에서 **가장 큰 액면의 동전을** 계속해서 선택한다.
  - ➢ 즉, 선택할 수 있는 여러 옵션 중에서 **지금 가장 최적으로 판단**되는 옵션을 선택하는 방법이다.
    - ▪ 이 때, 미래의 상황에 대해서는 고려하지 않는 것이 핵심.
  - ➢ 이러한 종류의 알고리즘을 "**그리디(Greedy) 알고리즘**"이라 한다.

안 통하는 경우가 많음!

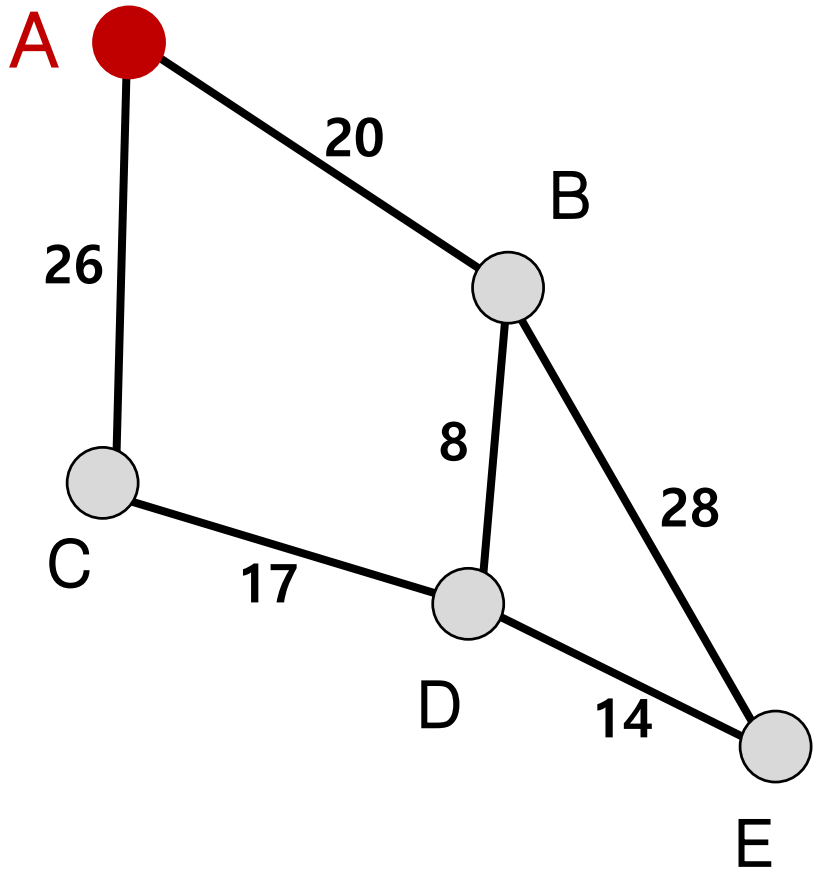
최단 경로 문제

- 특정 지점 A에서 출발하여 B에 도착하는 데 소요되는 최단 거리와 경로를 구하는 문제
  - ➢ 내비게이션과 같은 최적 경로 프로그램 구현 시 가장 기본이 되는 알고리즘
  - ➢ 실생활에서 가장 많이 쓰이는 알고리즘 중 하나

(사례) 금오공대 → 대구역 경로 탐색 결과



최단 경로 문제



- A에서 각 정점 간의 최단거리는?
  - ✓A → C: 26km
  - ✓A → B: 20km
  - ✓A → D: ?
    - A → C → D: 43km
    - A → B → D: 28km
  - ✓A → E: ?
    - A → B → E: 48km
    - A → B → D → E: 42km

최단 경로 문제

- 하나의 출발 정점에서 다른 나머지 정점들까지의 최단 거리를 구하는 문제
  - **다익스트라 알고리즘(Dijkstra's algorithm)**으로 해결
  - 대표적인 <u>그리디 알고리즘</u>

둘다 O(n^3)이지만 플로이드-와셜이 더 빠름

- 모든 정점 간의 최단 거리를 구하는 문제
  - **플로이드-와셜(Floyd-Warshall's algorithm)**으로 해결
  - 대표적인 <u>동적 계획 알고리즘</u> → 반복되는 중복계산 X
    EX. 귀납법, ★점화식

가짜 동전 찾기 문제

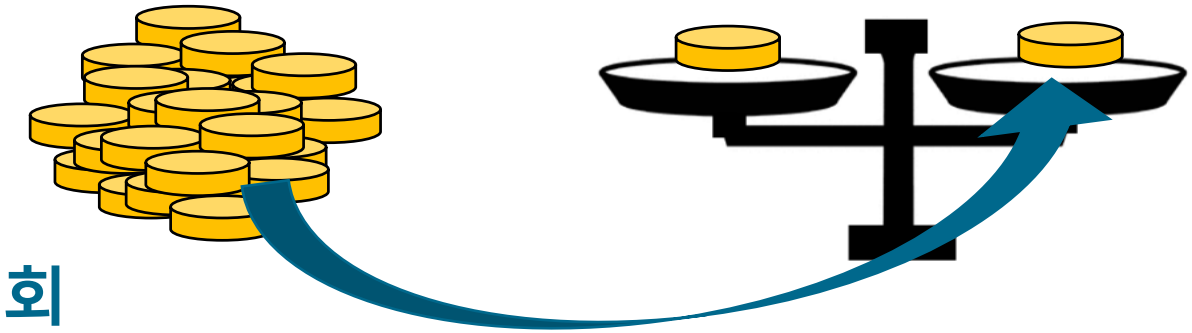
작년에 출제되었음!

- **문제 상황:** 아주 많은 동전 속에 1개의 가짜 동전이 있다.
  - ➢ 가짜 동전은 눈으로 식별할 수 없다.
  - ➢ 그러나 가짜 동전의 무게는 정상적인 동전보다 약간 가볍다.

- 가짜 동전 찾기 문제
  - ➢ **양팔 저울을 최소한의 횟수로 사용하여** 가짜 동전을 찾아내는 문제

가짜 동전 찾기 문제

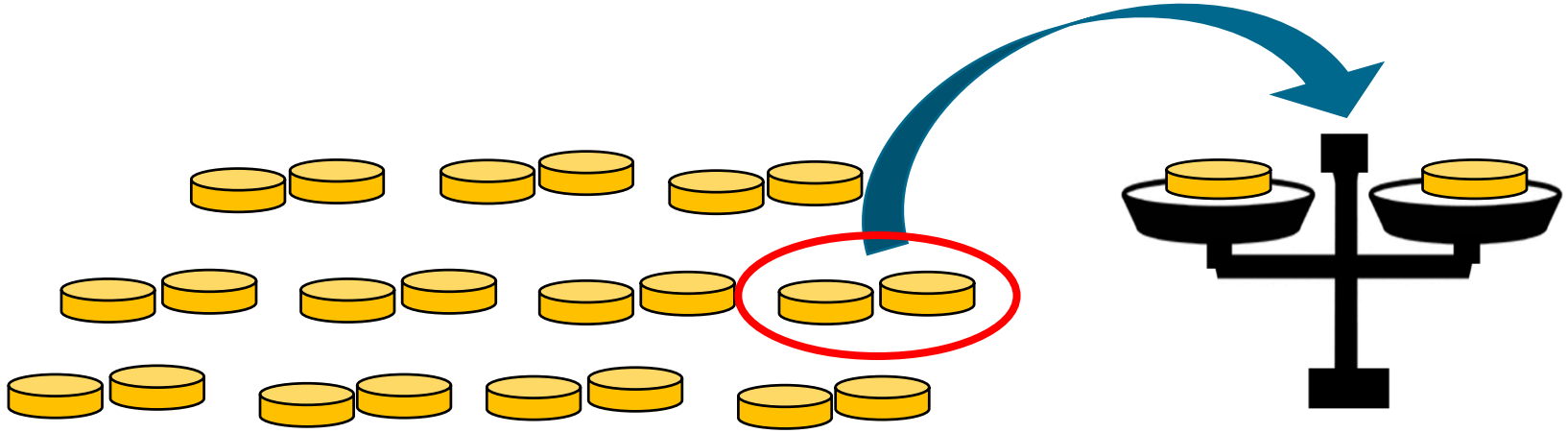
- **방법 1.** **동전 1개**를 저울 왼편에 올리고, 나머지 동전을 **하나씩** 저울 오른편에 올려서 가짜 동전을 찾아보자.



- 저울 사용 횟수: **최소 1회 ~ 최대 (n-1)회**
  - ➢ 운이 좋으면 1번 만에 가짜 동전을 찾는다.
  - ➢ 최악의 경우는 <u>가장 마지막에 올려 놓는 동전이 가짜 동전</u>인 경우
    - ➢ 이 경우, 총 동전의 개수가 n이라면, **(n-1)번 저울**을 재야 한다.

가짜 동전 찾기 문제

- **방법 2.** 동전을 **2개씩 짝을 지어**, n/2 짝을 각각 저울에 달아서 가짜 동전을 찾아보자.

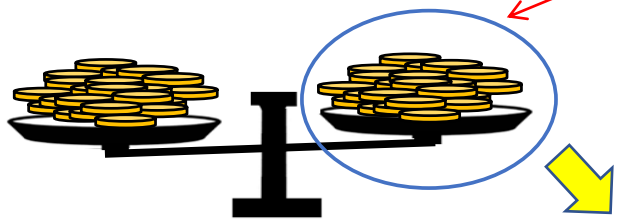


- 저울 사용 횟수: 최소 1회 ~ **최대 n/2회**
  - ➢ 최악의 경우 저울 사용 횟수가 약 1/2로 감소

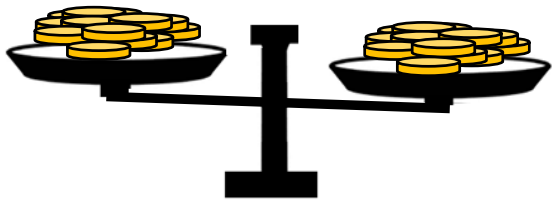
가짜 동전 찾기 문제

- **방법 3.** 동전 더미를 **반으로 나누어** 저울 양편에 놓기

가짜 동전이 있는 더미가 더 가벼움.
따라서 이쪽 더미에 가짜 동전이 있음.



이쪽 더미만 다시
반으로 나누어
무게를 측정해본다.



➢ 남은 동전 수는 계속 1/2로 줄어들고, 이 과정을 반복하여 최종적으로 2개의 동전이 남았을 때 가짜 동전을 가려낼 수 있다.

가짜 동전 찾기 문제

- 앞서 설명한 **"방법 3"**을 단계별로 기술하면…

  ① 동전 더미를 **반으로 나누어** 저울에 달고, **가벼운 쪽의 더미를 계속 반으로 나누어** 저울에 달아본다.

  ② 분할된 더미의 동전 개수가 1개씩이면 마지막으로 저울을 달았을 때, 가벼운 쪽의 동전이 가짜이다.

이 방법을 수행하면,

마지막 단계에서 가짜 동전을 찾을 수 있다.

가짜 동전 찾기 문제

- 만약 1024개의 동전이 주어진 경우, "방법 3"을 수행하면 저울을 몇 번 달아야 할까?
  - 먼저 512개씩 저울 양쪽에 올려놓고 무게를 측정하고,
  - 그 다음은 256개씩 저울 양쪽에 올려놓고 무게를 측정하고,
  - 그 다음은 128개씩, 64개씩, 32개씩, 16개씩, 8개씩, 4개씩, 2개씩,
  - 마지막엔 1개씩 저울 양쪽에 올려서 무게를 측정한다.
  - 이는 총 10번이고, 이 횟수를 동전의 개수와의 관계식으로 표현하면 $\log_2 1024 = \log_2 2^{10} = 10$ 이다.

∴ n개의 동전이 주어졌을 때, 총 $\log_2 n$번 저울에 달아 무게를 측정하면, 가짜 동전을 찾을 수 있다.

가짜 동전 찾기 문제

- "방법 1"은 **최악의 경우** 1,023번 저울에 달아야 한다.
- "방법 2"는 **최악의 경우** 512번 달아야 한다.
- "방법 3"은 **최악의 경우** 10번이면 가짜 동전을 찾는다.

이 차이는 동전의 개수(=n)가 많아질수록 더욱 커진다.

가짜 동전 찾기 문제

- 가짜 동전 찾기 문제의 어려움 → 동전의 개수가 너무 많음.
- 이와 같이, 문제의 크기가 매우 커서 직접적으로 문제 해결이 어려운 경우
  ① 주어진 문제를 더욱 작은 크기의 부분 문제로 나눈다.
  ② 각 부분 문제에 대하여 해답을 찾는다.
  ③ 각 부분 문제에 대하여 찾은 해답을 조합하여 전체 문제의 해답을 찾는다.

- 이와 같은 방법으로 문제를 해결하는 알고리즘을 "**분할 정복 알고리즘 (Divide & Conquer)**"이라 한다.
  by. 나폴레옹

Q & A